

Compete Agent

Product Requirements Document

Minimum Viable Product (MVP)

Prepared by	OneGTM Lab
Prepared for	Litera
Authors	Antony, Ajay
Status	DRAFT v2.0 — for client review & sign-off
Purpose	Validate our understanding and the delivered MVP
Date	13 July 2026

Document Control

Field	Detail
Document	Compete Agent — Product Requirements Document (MVP)
Prepared by	OneGTM Lab
Prepared for	Litera
Purpose	Present our understanding and the delivered MVP for Litera's validation
Status	DRAFT v2.0 — for client review and sign-off
Date	13 July 2026
Classification	Confidential — Litera & OneGTM Lab

How to read this document

This PRD does two things at once:

1. **States our understanding** of the problem, users, and workflow — so Litera can confirm or correct it.
2. **Describes what the MVP already delivers**, feature by feature, with an honest status on each — so Litera can judge whether we are building the right thing, the right way.

Items marked **[NEEDS LITERA]** are the specific inputs or decisions we need from you. Section 16 collects them all in one checklist.

1. Executive Summary

Litera already captures competitive intelligence well — Crayon, Gong, Salesforce, and frontline conversations produce a steady stream of signals. What is slow and manual is everything after capture: a Product Marketing Manager (PMM) must read each signal, decide who needs it, and hand-write different versions for Sales, Product, Marketing, and Leadership — often days after the competitor moved.

Compete Agent automates that layer. One competitive signal goes in; classified, interpreted, audience-routed, team-ready content comes out — for all four teams at once — with a **mandatory human review gate enforced in the database** before anything is published, and delivery into **Microsoft Teams and email** where teams already work.

The MVP is built and running end-to-end today. Its defining engineering choice is **trust before fluency**:

- The agent **states only what the source signal and confirmed competitor facts support**. Anything beyond that is flagged, never asserted.
- The agent currently **claims nothing about Litera at all** — the Litera fact allowlist ships deliberately empty until Litera supplies approved facts. It cannot invent a certification, feature, or differentiator.
- Every generated output passes an **automated 8-rule quality gate** plus deterministic safety checks; if content cannot be verified clean, the signal errors visibly rather than shipping bad copy. The design rule is *clean-or-error, no silent bad copy*.
- Nothing reaches a human outside the review team without a **reviewer's explicit approval**, enforced by row-level security in Postgres — a direct API call cannot bypass it.

This document lays out what we understood, what we built, what is gated on Litera inputs, and the specific questions we need answered to confirm we are on the right track.

2. Our Understanding of the Problem — please validate

2.1 Current-state workflow (as we understand it)

1. A competitive signal surfaces in Crayon, Gong, Salesforce, or a call/email.
2. A PMM notices it — often late — and reads through it.
3. The PMM decides which teams care and what each should do.
4. The PMM manually writes separate content for each team.
5. The PMM distributes by copy-paste into Teams, email, or decks.
6. Teams receive it at different times, in inconsistent formats.

2.2 Pain points we designed against

#	Pain point	Consequence
P1	Signals fragmented across tools	No single view of "what changed and what it means"
P2	Content rewritten manually per team	Slow, unscalable; the PMM is the bottleneck
P3	Tone and quality vary by author	Off-brand messaging reaches customers
P4	Teams enabled at different times	GTM response is late and uncoordinated
P5	Generic AI tools fabricate claims	Risk of unverified intel reaching the field
P6	No audit trail of what was approved	Hard to govern what the field is told

[NEEDS LITERA] Does this match reality? Are there pain points we are missing or over-weighting?

3. Product Principles (non-negotiable in the build)

1. **Human-in-the-loop always.** Nothing publishes without explicit reviewer approval — enforced at the database layer, not just the interface.
2. **Grounded, never invented.** Only what the source signal or confirmed competitor facts support is stated; the rest is flagged or removed.
3. **Assume nothing about Litera.** Until Litera supplies an approved fact list, the agent makes zero claims about Litera's products, certifications, or capabilities.
4. **Meet teams where they work.** Delivery is Microsoft Teams and email — no new tool for recipients to adopt.
5. **On top of, not instead of.** Compete Agent complements Crayon, Gong, and Salesforce; it does not replace them.
6. **Litera controls the model.** The AI backend is selectable per deployment, so Litera governs where its competitive data is processed.

4. Users & Roles

4.1 Personas

Persona	Role in the product	What they get
PMM (primary user)	Reviewer — reviews, edits, approves, publishes	One queue; one-click enablement of every team
Sales / CS	Recipient	Talk track, battlecard, live talking points
Product Management	Recipient	Watch-outs with roadmap implications
Marketing	Recipient	Campaign angles when a competitor shifts
Executive Leadership (ELT)	Recipient	Concise strategic summaries

Recipients do not log in — they consume content in Teams/email. Only the PMM (and any admins) needs an account.

4.2 Roles as implemented

Three roles exist in the data model, enforced by row-level security: **submitter** (submit signals only), **reviewer** (approve, edit, publish, manage competitor facts), **admin** (all of the above plus connector management). New accounts default to submitter. In the current MVP the UI shows all screens to any logged-in user; the *database* enforces who can actually approve and publish. Role-tailored navigation is a planned hardening item.

[NEEDS LITERA] Who are the named reviewers? Is a single reviewer (the PMM) correct for Phase 1, or do you need multiple reviewers or a rotating role?

5. The End-to-End Workflow (as built)

1. **Signal arrives** — pasted manually at /intake, pushed by a Crayon webhook, or pulled by a scheduled connector fetch.
2. **Classify** — the agent identifies the competitor (matched against Litera's competitor list), signal type, business area, urgency (low/medium/high), and per-team relevance.
3. **Interpret** — a grounded plain-language summary and "why it matters to Litera," with any unsupported inference flagged as an unverified claim rather than asserted.
4. **Route** — deterministic rules (no AI): high relevance auto-queues an output for that team; medium queues it flagged for reviewer discretion; low generates nothing.
5. **Package** — team-specific content is generated per routed team (six output types across four teams), then passed through the automated quality gate and safety checks (Section 7).
6. **Human review** — the reviewer reads, edits if needed, and approves or rejects each output in the review queue. Approval is blocked at the database until any flagged claim is resolved.

- Deliver** — approved outputs post to a Microsoft Teams channel as an urgency-colored card and/or go out by email; a weekly digest batches everything approved since the last send.

A full pipeline run completes in well under a minute on a hosted model. Duplicate submissions of identical text are detected and not re-processed; failed runs are marked visibly and can be retried in one click.

6. What the MVP Delivers — feature status

Legend: ■ built and working today · ■ built, needs a Litera input to go live · ■ planned next.

6.1 Signal intake

Feature	Status	Notes
Manual intake form (paste text, source type, optional source link)	■	Chat-style composer
De-duplication by content hash	■	Identical text is not re-processed; an errored duplicate is automatically re-run
Manual "Process" control for pulled signals	■	Crayon-pulled signals wait for a click, keeping AI usage controlled
Retry for failed or stuck signals	■	"Needs attention" panel flags errors and mid-pipeline stalls (>5 min) with one-click retry

6.2 Intelligence pipeline

Feature	Status	Notes
Classification (competitor, type, business area, urgency, per-team relevance)	■	Competitor matched against Litera's managed list, or "Unknown"
Grounded interpretation with unverified-claim flagging	■	Grounding rule embedded in every prompt
Deterministic audience routing	■	high → auto, medium → reviewer discretion, low → skipped
Per-team packaging — 6 output types across 4 teams	■	Sales: talk track, battlecard, live talking points · Product: watch-out · Marketing: campaign angle · Leadership: summary
Automated quality gate + safety checks	■	Section 7 — every output, every signal, no skip path
Schema validation on every AI response, with one guided retry	■	Malformed output never reaches the database
Measured tone on early signals	■	A product page or demo is framed as a signal to watch, not a proven threat

6.3 Human review & approval

Feature	Status	Notes
Review queue sorted by urgency, with stat tiles	■	Pending / High / Medium / Low / Unverified counts
Edit before approval (edits tracked for quality metrics)	■	Edit count and timestamp feed the analytics
Approve / Reject per output	■	Approver identity and timestamp recorded

Feature	Status	Notes
Database-enforced approval gate	■	Row-level security blocks approving any output carrying an unresolved claim — even via direct API call
Claim resolution (confirm into content, or remove)	■	API is built; the review-screen control for it is being surfaced in the current iteration

6.4 Delivery

Feature	Status	Notes
Publish to Microsoft Teams	■	Built and working; needs Litera's channel webhook URL(s). Card shows urgency color, competitor, source, approver, date
Per-output email (recipient chosen at send time)	■	Transactional email; approval required; can re-send to additional recipients
Weekly/bi-weekly email digest	■	Batches everything approved since last send into one email; scheduled run every Monday, plus manual "Send now"
Re-publish protection	■	An output cannot be posted to Teams twice; first channel to publish stamps the record

6.5 Source connectors

Connector	Status	Notes
Crayon — webhook push (preferred)	■	Fully built with shared-secret authentication and auto-processing; needs Litera's Crayon plan to enable outbound webhooks
Crayon — daily API poll (alternative)	■	Fully built (daily scheduled fetch); needs Litera's Crayon API key
Salesforce — inbound	■	Live code complete (config-driven field mapping, never hardcoded); gated on Litera's Salesforce admin confirming object/field names and providing credentials
Gong — inbound	■	Live code complete (7-day call window); gated on confirming API access on Litera's Gong plan tier
Test Mode for all three inbound connectors	■	Each runs against realistic fixture data today, clearly badged Test Mode — the full pipeline is exercised without any live credential

6.6 Insight surfaces & administration

Feature	Status	Notes
Dashboard of published outputs	■	Filter by team, competitor, date range
Quality analytics	■	Approval rate without edits, edit rate, average review time, totals — overall and per output type

Feature	Status	Notes
Competitor management with known-facts	■	Facts ground the interpretation stage; add/remove per competitor with source labels
Connector settings with credential vaulting	■	Secrets stored in Supabase Vault, resolved server-side only; the browser can never read a stored secret
AI provider settings	■	Paste one API key — provider auto-detected; or pin a specific model; or automatic fallback chain
Scheduled automation	■	Three scheduled jobs (daily Crayon poll, 30-minute inbound poll, weekly digest) secured by a shared secret

7. The Trust Layer (what makes this safe to put in front of the field)

This is the part of the build we most want Litera to scrutinize.

Layer 1 — Grounding at generation. Every prompt that produces audience-facing content carries the rule: state as fact only what the raw signal or the competitor's confirmed known-facts support; everything else is an unverified claim, not an assertion.

Layer 2 — The empty Litera allowlist. The system holds an allowlist of confirmed facts about Litera itself, and it currently ships **deliberately empty**. Every generation and verification prompt therefore carries a hard notice: *do not claim, imply, or invent any Litera capability, certification, product, feature, pricing, or security posture*. Where a Litera fact would be needed, the content instead plainly reports what the customer asked or what the competitor did, so the rep decides. **[NEEDS LITERA]** — supply the approved fact list (e.g. certifications, flagship capabilities) and the agent will use exactly those, nothing more.

Layer 3 — The automated quality gate. After drafting, all outputs pass a single verification step enforcing eight rules: grounding; Litera-allowlist only; never attribute a competitor's feature to Litera; marketing content never names the customer account; battlecards must lead with a competitor weakness the signal actually revealed; no generic filler; only the signal's competitor may be named (all other competitors are explicitly banned from the text); and only the sales talk track may address the customer as "you."

Layer 4 — Deterministic safety checks. Non-AI code then verifies: any customer/account name is stripped from marketing content; contradictory product watch-outs are corrected; if any *other* competitor's name survived, or an output is substantially a verbatim copy of the source, the whole signal **errors visibly instead of publishing** — *clean-or-error, no silent bad copy*.

Layer 5 — The human gate. Finally, a reviewer must approve. The database itself refuses to set an output approved while flagged claims remain — a policy in Postgres row-level security, meaning it holds even against direct API calls that bypass the UI entirely.

8. AI Model Strategy

The pipeline is provider-agnostic behind a single interface, with structured-output validation on every call regardless of backend:

Backend	Role
Claude (Anthropic)	Recommended production model — strongest instruction-following for the grounding rules
Gemini (Google)	Supported hosted alternative; auto-detected from the key format

Backend	Role
Ollama (self-hosted, two-tier)	Development/testing on local or LAN hardware — data never leaves the network
Cloudflare Workers AI	Low-cost testing backend

An admin pastes one API key on the settings page (stored in Supabase Vault, never shown again, never sent to the browser) or pins a specific model. Default behavior is an automatic chain: hosted API first, then network model, then local model. Grounding behavior is production-trusted only after the dedicated grounding test passes on the chosen hosted model.

[NEEDS LITERA] Confirm the production model (we recommend Claude) and any data-residency constraints.

9. Delivery Channels (as built)

Microsoft Teams. Approved outputs post to a channel via an Incoming Webhook as a formatted card: urgency-colored bar, audience and output type, competitor, source, approver, and date. One webhook per channel; no bot registration or Azure AD consent required. Known limitation, by design of Teams webhooks: they are one-way — approve/reject cannot happen inside Teams; all review stays in the web app. (In-Teams approval would require a registered Teams bot via the Graph API — out of MVP scope.)

Email — two modes. (1) Per-output send: the reviewer types/confirms a recipient at publish time; approval is required first. (2) Digest: everything approved since the last send is batched into one email on a weekly or bi-weekly cadence (Litera's choice), fired by a scheduled job each Monday or manually on demand. Email open-tracking is not built — SMTP does not support it reliably; adoption measurement is a Phase-3 item.

[NEEDS LITERA] Which Teams channels map to which teams, and which distribution lists should receive email?

10. Data Model & Security (summary)

Entity	Purpose
competitors	Competitor list + confirmed known-facts that ground interpretation
signals	Raw signals, source, dedupe hash, pipeline status
signal_classification	Competitor, type, urgency, per-team relevance
signal_outputs	Per-team content, claims, approval state, edit tracking, publish/digest stamps
user_profiles	Users and roles (submitter / reviewer / admin), auto-created on signup
connectors	Configured integrations, mode (test/live), status, cadence
llm_config	Active AI provider/backend selection

Signal lifecycle: draft → classified → interpreted → packaged → published, with rejected and error as visible branch states.

Security posture as implemented:

- All connector and AI credentials live in **Supabase Vault**; decryption functions are executable only by the backend service role — the browser can never resolve a secret, even with its reference ID.
- Row-level security is enabled on **every table**. Pipeline writes go through the backend only; clients cannot alter pipeline state.
- The approval gate and the reviewer/admin write restrictions are RLS policies — server-enforced, not UI conventions.

- Scheduled jobs and the Crayon webhook authenticate with dedicated shared secrets.
- Every published output records who approved it and when — a full audit trail of what the field was told.
- A data-handling policy governs live customer-adjacent data (Gong transcripts, Salesforce notes): production AI is the hosted approved model only, minimal fields are sent, and each connector requires sign-off before going live.

11. Analytics & Measurement (as built)

The analytics page reports, overall and per output type: **approval rate without edits** (the clearest proxy for content quality), **edit rate**, **average review time** (queue throughput), and totals for approved / rejected / pending / published. These are the metrics that tell Litera whether the agent's content is actually good enough — from day one.

[NEEDS LITERA] Today's baselines (time from signal to teams enabled; PMM hours per signal; signals per week) so we can quantify the improvement.

12. Explicitly Out of Scope (MVP)

- Autonomous publishing without a human approval.
- Public web crawling / open-web monitoring (Compete Agent works from Litera's trusted sources).
- Replacing Crayon, Gong, or Salesforce.
- In-Teams approval (requires a registered bot — Phase 2+ candidate).
- Read-only logins for recipient teams.
- Slack delivery.
- Email open/click tracking.

13. Known Limitations — stated plainly

We would rather Litera hear these from us than find them:

1. **Salesforce and Gong live modes are code-complete but unproven against Litera's instances** — field mappings and API access must be confirmed before first live fetch. Test Mode fully exercises the pipeline meanwhile.
2. **Salesforce authentication currently uses a static token** — a token-refresh flow is a hardening item before sustained live use.
3. **Teams webhooks are one-way** — no in-Teams approval, and Microsoft is migrating webhook creation toward Power Automate "Workflows"; our integration posts to a URL and works with either mechanism.
4. **UI is not yet role-tailored** — the database enforces permissions; the interface showing all controls to all logged-in users is a polish item, as are sign-out and the claim-resolution control on the review screen.
5. **Adoption measurement is deferred** — Teams reactions can't be read via webhooks and SMTP has no open tracking; Phase 3 addresses this.
6. **Brand voice is not yet tuned to Litera** — deliberately. The trust rules are in place; the tone layer waits on Litera's messaging guidelines (Section 16), which slot directly into the generation prompts.

14. Acceptance Criteria — how Litera can verify the MVP

Run this checklist against the live system:

1. Submit a signal → it classifies, interprets, routes, and packages automatically, visibly moving through each status.
2. Outputs exist for each relevant team, in that team's format — and none for low-relevance teams.
3. Feed a signal containing a deliberate overreach ("this means X has fundamentally better AI") → the overreach is flagged/removed, not asserted.

4. Confirm no output states any fact about Litera (empty allowlist behavior), and no output names a second competitor.
5. Attempt to publish an unapproved output via direct API call → blocked.
6. Approve an output → it posts to the Teams channel and/or emails the chosen recipient; approver and timestamp are recorded.
7. Send the digest → one batched email of everything approved since last send; items never re-send.
8. Filter the dashboard by team/competitor/date → correct published outputs return.
9. Kill a pipeline mid-run → the signal appears in "Needs attention" and retries cleanly.

15. Delivery Phases

Phase	Outcome	Status
MVP (this document)	Full pipeline + trust layer + review gate + Teams/email delivery + Test-Mode connectors + analytics	Delivered — pending Litera validation & inputs
Phase 2 — Go live	Live Crayon/Salesforce/Gong feeds; Litera facts + brand voice tuned; data-handling sign-off per connector	Gated on Section 16 inputs
Phase 3 — Measure & refine	Adoption metrics, prompt refinement loop driven by approval/edit rates, role-tailored UI	Planned

Commercials and timeline sit in a separate SOW.

16. What We Need From Litera — the validation checklist

Decisions

#	Decision	Blocks
D1	Confirm the problem statement and the 7-step PMM workflow match reality (Sections 2, 5)	Everything
D2	Named reviewer(s) — single PMM owner or multiple?	Roles, go-live
D3	Production AI provider (we recommend Claude) + any data-residency constraints	Phase 2
D4	Digest cadence: weekly or bi-weekly	Digest default
D5	Which connector goes live first	Phase 2 ordering

Inputs

#	Input	Feeds
I1	Approved Litera fact list — certifications, flagship capabilities, anything the agent may state about Litera	The (currently empty) allowlist — Section 7
I2	Brand tone / messaging guidelines	Generation prompts — Section 13.6
I3	Competitor list + confirmed known-facts per competitor	Classification & grounding
I4	Salesforce object/field names (from Litera's SF admin) + credentials	Salesforce live mode

#	Input	Feeds
I5	Gong API access confirmation (plan tier) + credentials	Gong live mode
I6	Crayon API key, or webhook enablement	Crayon live mode
I7	Teams channel webhook URL(s) per team	Teams delivery
I8	Email distribution lists per team; verified sender domain	Email delivery
I9	Baseline metrics (time-to-enablement, PMM hours/signal, signals/week)	Success measurement

The core question we are asking with this document: does the workflow, the trust posture, and the feature set above match what Litera needs a PMM to do? Where it does not, tell us — that is precisely what this review is for.

17. Appendix

17.1 Output type reference

Output	Team	What it is
Talk track	Sales	One confident, grounded line a rep can say to a customer
Battlecard	Sales	Objection → response pair, leading with the competitor's signal-revealed weakness
Live talking points	Sales	1–2 minute spoken-style briefing for a team meeting
Watch-out	Product	The rival capability and its roadmap implication — or an explicit "no product implication" call
Campaign angle	Marketing	Market-positioning angle; never names the customer account
Leadership summary	Leadership	2–3 sentence executive "so what"

17.2 Glossary

Term	Meaning
Signal	A raw piece of competitive intelligence (alert, call note, news, website change)
Grounding	Restricting output to what the source and confirmed facts support
Unverified claim	A statement beyond the source; flagged for the reviewer, never asserted
Quality gate	The automated post-generation verification pass (8 rules + safety checks)
Allowlist	The set of approved Litera facts the agent may state — currently empty by design
Reviewer	The role permitted to approve and publish (the PMM in the MVP)

17.3 Technology summary

Next.js + Tailwind frontend · Supabase (Postgres, Auth, Row-Level Security, Vault) · provider-agnostic AI layer (Claude / Gemini / Ollama / Cloudflare) with schema-validated structured output · Microsoft Teams Incoming Webhooks · transactional + digest email · scheduled jobs via GitHub Actions · adapter-pattern connectors (Crayon, Gong, Salesforce) with Test/Live modes.

Prepared by OneGTM Lab for Litera. This document reflects the system as actually built and is shared for validation. It does not constitute a final scope or commercial commitment; commercial terms sit in a separate SOW.